

1. Develop a C program that demonstrates how a Linux operating system executes a command entered by a user that 1. Accept a Linux command as input. 2. Create a child process using fork(). 3. Execute the command in the child process using an appropriate exec() system call. 4. Allow the parent process to wait for the child using wait (). 5. Display the Process ID (PID) of both parent and child processes.

Using Linux terminal commands (uname, lscpu, lsblk, ps, top), investigate the relationship between hardware resources and operating system services. Prepare a report explaining how the OS abstracts CPU, memory, storage, and I/O devices.

```
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>
#include <sys/wait.h>
int main(){
char command[100];
printf("Enter a linux command: ");
scanf("%s",command);
int pid;
pid = fork();
if(pid==0){
printf("Child PID: %d\n",getpid());
execlp(command,command,NULL);
}else
{
printf("Parent PID: %d\n",getpid());
wait(NULL);
}
return 0;
}
```

```
prudhvi@prudhvisai-06:~$ nano practical1.c
prudhvi@prudhvisai-06:~$ gcc practical1.c
prudhvi@prudhvisai-06:~$ ./a.out
Enter a linux command: ls
Parent PID: 676
Child PID: 677
OSLAB          practical2.c    process        program4.c
a.out          practical3.c    process.c      program5.c
copy.txt       practical4.2.c  process.c.save program6.co
dest.txt       practical4.c    process.c.save.1 program9.c.save
fcfs.c         practical5.2.c  program.c1     prudhvi
file1.txt      practical5.c    program.cooooo robin.c
hello.c.save   practical6.2.c  program1.c     roundrobin.c
o              practical6.c    program1.co    sample.txt
oslab          pratical1.c    program2.c     sjf.c
practical1.c   practical2.c    program3.c
prudhvi@prudhvisai-06:~$
```

2. Develop a C program that uses the system calls `open()`, `read()`, `write()`, and `close()` to copy the contents of one file to another. Explain how control transitions between user space and kernel space during execution.

Use the `strace` utility to trace all system calls generated by the command `cat sample.txt`. Analyze the sequence of system calls and identify the kernel services involved.

```
#include<stdio.h>
#include<unistd.h>
#include<fcntl.h>
int main(){
char buffer[1024];
int n;
int src = open("source.txt", O_RDONLY);
int dest = open("dest.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
while ((n = read(src, buffer, 1024)) > 0) {
write(dest, buffer, n);
}
close(src);
close(dest);
printf("File copy complete.\n");
return 0;
}
```

```
prudhvi@prudhvisai-06:~$ nano practical2.c
prudhvi@prudhvisai-06:~$ gcc practical2.c
prudhvi@prudhvisai-06:~$ ./a.out
File copy complete.
prudhvi@prudhvisai-06:~$
```

3. Develop a C program using fork() that creates a parent and child process. Display the Process ID (PID), Parent Process ID (PPID), and process states at different stages of execution.

Design an experiment to observe process state transitions (Ready, Running, Waiting, Terminated) using Linux monitoring tools such as ps, top, and /proc. Document the observations.

```
#include<stdio.h>
#include<stdlib.h>
#include<unistd.h>
#include<sys/wait.h>
int main(){
int pid = fork();
if(pid<0){
printf("Fork Failed\n");
}
else if (pid==0){
printf("Child PID: %d\n",getpid());
printf("Parent PID: %d\n",getppid());
sleep(10);
}
else{
printf("Parent PID: %d\n",getpid());
sleep(10);
}
return 0;
}
```

```
prudhvi@prudhvisai-06:~$ nano practical3.c
prudhvi@prudhvisai-06:~$ gcc practical3.c
prudhvi@prudhvisai-06:~$ ./a.out
Parent PID: 815
Child PID: 816
Parent PID: 815
```

4. Write a C program where a parent process creates multiple child processes and synchronizes their completion using `wait()` and `waitpid()`. Compare the behavior of both functions.

Create a scenario where a child process becomes a zombie process. Investigate the process table and then modify the program to eliminate zombie processes using proper synchronization techniques.

```
#include<stdio.h>
#include<unistd.h>
#include<sys/wait.h>
int main()
{
    int p1, p2, p3;
    p1 = fork();
    if (p1 == 0)
    {
        printf("Child 1 PID: %d\n", getpid());
        return 0;
    }
    p2 = fork();
    if (p2 == 0)
    {
        printf("Child 2 PID: %d\n", getpid());
        return 0;
    }
    p3 = fork();
    if (p3 == 0)
    {
        printf("Child 3 PID: %d\n", getpid());
        return 0;
    }
    waitpid(p2, NULL, 0);
    printf("Parent: Child 2 completed\n");
    return 0;
}
```

```
prudhvi@prudhvisai-06:~$ nano practical4.c
prudhvi@prudhvisai-06:~$ gcc practical4.c
prudhvi@prudhvisai-06:~$ ./a.out
Child 1 PID: 865
Child 2 PID: 866
Child 3 PID: 867
Parent: Child 2 completed
prudhvi@prudhvisai-06:~$
```

```
#include<stdio.h>
#include<unistd.h>
#include<stdlib.h>
#include<sys/wait.h>
int main(){
if(fork()==0){
printf("Child process\n");
printf("CP PID is=%d\n",getpid());
printf("CP exeution is done\n");
exit(0);
}else{
printf("Parent process\n");
printf("pp pid is = %d\n",getpid());
sleep(10);
printf("PP done");
}
return 0;
}
```

```
prudhvi@prudhvisai-06:~$ nano practical4.2.c
prudhvi@prudhvisai-06:~$ gcc practical4.2.c
prudhvi@prudhvisai-06:~$ ./a.out
Parent process
pp pid is = 910
Child process
CP PID is=911
CP exeution is done
PP doneprudhvi@prudhvisai-06:~$ |
```

5. Implement a producer-consumer communication system using anonymous pipes where the parent process generates data and the child process consumes it. Measure the communication efficiency.

Develop a program that executes the equivalent of the shell command: `ls -l | grep ".c"` using `fork()`, `pipe()`, `dup2()`, and `exec()` system calls.

```
#include<stdio.h>
#include<unistd.h>
#include<sys/wait.h>
int main(){
int fd[2];
pid_t p1,p2;
pipe(fd);
p1=fork();
if(p1==0){
close(fd[0]);
dup2(fd[1],STDOUT_FILENO);
close(fd[1]);
execvp("ls","ls","-l",NULL);
}
p2 = fork();
if(p2==0){
close(fd[1]);
dup2(fd[0],STDIN_FILENO);
close(fd[0]);
execvp("grep","grep",".c",NULL);
}
close(fd[0]);
close(fd[1]);
wait(NULL);
wait(NULL);
return 0;
}
```

```

prudhvi@prudhvisai-06:~$ nano practical5.c
prudhvi@prudhvisai-06:~$ gcc practical5.c
prudhvi@prudhvisai-06:~$ ./a.out
-rw-r--r-- 1 prudhvi prudhvi      0 Sep 13 09:33 copy.txt
-rw-r--r-- 1 prudhvi prudhvi    382 Sep  1 03:18 fcfs.c
-rw----- 1 prudhvi prudhvi      13 Jul 28 04:21 hello.c.save
-rw-r--r-- 1 prudhvi prudhvi    338 Sep 14 11:01 practical1.c
-rw-r--r-- 1 prudhvi prudhvi    338 Sep 14 11:04 practical2.c
-rw-r--r-- 1 prudhvi prudhvi    318 Sep 14 11:07 practical3.c
-rw-r--r-- 1 prudhvi prudhvi    324 Sep 14 11:11 practical4.2.c
-rw-r--r-- 1 prudhvi prudhvi    398 Sep 14 11:09 practical4.c
-rw-r--r-- 1 prudhvi prudhvi    405 Sep 13 15:18 practical5.2.c
-rw-r--r-- 1 prudhvi prudhvi    389 Sep 14 11:12 practical5.c
-rw-r--r-- 1 prudhvi prudhvi    394 Sep 14 10:57 practical6.2.c
-rw-r--r-- 1 prudhvi prudhvi    295 Sep 13 16:16 practical6.c
-rw-r--r-- 1 prudhvi prudhvi   2009 Aug 11 03:34 pratical1.c
-rw-r--r-- 1 prudhvi prudhvi   1995 Aug 11 03:30 pratical2.c
-rwxr-xr-x 1 prudhvi prudhvi  16048 Aug  4 04:09 process
-rw-r--r-- 1 prudhvi prudhvi    160 Aug  4 03:44 process.c
-rw----- 1 prudhvi prudhvi     19 Aug  4 03:17 process.c.save
-rw----- 1 prudhvi prudhvi      9 Aug  4 03:18 process.c.save.1
-rw-r--r-- 1 prudhvi prudhvi    159 Aug  4 03:09 program.c1
-rw-r--r-- 1 prudhvi prudhvi    159 Aug  4 03:06 program.coooooooo
-rw-r--r-- 1 prudhvi prudhvi    161 Aug  4 03:33 program1.c
-rw-r--r-- 1 prudhvi prudhvi    161 Aug  4 03:29 program1.co
-rw-r--r-- 1 prudhvi prudhvi    189 Aug  4 03:48 program2.c
-rw-r--r-- 1 prudhvi prudhvi    213 Aug  4 03:53 program3.c
-rw-r--r-- 1 prudhvi prudhvi    149 Aug  4 04:01 program4.c
-rw-r--r-- 1 prudhvi prudhvi    197 Aug  4 04:08 program5.c
-rw-r--r-- 1 prudhvi prudhvi    144 Aug 11 03:06 program6.co
-rw----- 1 prudhvi prudhvi    249 Aug  7 04:54 program9.c.save
-rw-r--r-- 1 prudhvi prudhvi      0 Sep  1 03:45 robin.c
-rw-r--r-- 1 prudhvi prudhvi    617 Sep  1 03:48 roundrobin.c
-rw-r--r-- 1 prudhvi prudhvi    502 Sep  1 03:28 sjf.c
prudhvi@prudhvisai-06:~$

```

```
#include<stdio.h>
#include<unistd.h>
#include<string.h>
#include<fcntl.h>
int main(){
int fd[2];
pid_t pid;
char data[] = "Hello Producer";
char buffer[100];
pipe(fd);
pid = fork();
if(pid==0){
close(fd[1]);
read(fd[0],buffer,sizeof(buffer));
printf("Consumer received: %s\n",buffer);
close(fd[0]);
}
else{
close(fd[0]);
write(fd[1],data,strlen(data)+1);
printf("Producer sent: %s\n",data);
}
return 0;
}
```

```
prudhvi@prudhvisai-06:~$ nano practical5.2.c
prudhvi@prudhvisai-06:~$ gcc practical5.2.c
prudhvi@prudhvisai-06:~$ ./a.out
Consumer received: Hello Producer
Producer sent: Hello Producer
prudhvi@prudhvisai-06:~$
```

6. Create a client-server application using Named Pipes (FIFOs). The client sends messages and the server processes and responds to them. Analyze FIFO behavior when multiple clients communicate simultaneously.

Create a POSIX signal handling program that captures SIGINT, SIGTERM, and SIGUSR1. Demonstrate asynchronous event handling and explain the role of signal handlers.

```
#include<stdio.h>
#include<fcntl.h>
#include<unistd.h>
#include<string.h>
#include<sys/stat.h>
int main(){
int fd;
char message1[] = "Hello world";
fd = open("/tmp/myfifo", O_WRONLY);
write(fd,message1,strlen(message1)+1);
printf("Client sent the message: %s\n",message1);
close(fd);
return 0;
}
```

```
#include<stdio.h>
#include<signal.h>
void handler(int sig)
{
if(sig == SIGINT)
printf("SIGINT received\n");
if(sig == SIGTERM)
printf("SIGTERM received\n");
if(sig == SIGUSR1)
printf("SIGUSR1 received\n");
}
int main()
{
signal(SIGINT, handler);
signal(SIGTERM, handler);
signal(SIGUSR1, handler);
printf("Signal handling program\n");
raise(SIGINT);
raise(SIGTERM);
raise(SIGUSR1);
return 0;
}
```

```
prudhvi@prudhvisai-06:~$ nano practical6.c
prudhvi@prudhvisai-06:~$ nano practical6.2.c
prudhvi@prudhvisai-06:~$ gcc practical6.2.c
prudhvi@prudhvisai-06:~$ ./a.out
Signal handling program
SIGINT received
SIGTERM received
SIGUSR1 received
prudhvi@prudhvisai-06:~$
```
